

Adaptive Markov Chain Monte Carlo Introduction

Gabriel Alwan

Why Do We Need Sampling?

- Bayesian inference requires computing the posterior $p(\theta|y) = \frac{p(y|\theta)p(\theta)}{\int p(y|\theta)p(\theta)d\theta}$.
- The denominator $Z = \int p(y|\theta)p(\theta)d\theta$ is the **normalizing constant**. In most realistic models this integral is computationally intractable.
- We can still evaluate the unnormalized density $f(\theta) = p(y|\theta)p(\theta)$ so that $p(\theta|y) \propto f(\theta)$.
- Many quantities of interest are expectations such as $E[g(\theta)|y] = \int g(\theta)p(\theta|y)d\theta$.
- If we can generate samples $\theta_1, \dots, \theta_N \sim p(\theta|y)$ then $E[g(\theta)] \approx \frac{1}{N} \sum_{i=1}^N g(\theta_i)$.
- Direct methods (i.e. grid approximation) scale poorly with dimension, motivating Monte Carlo methods.

Markov Chain Monte Carlo (MCMC)

- Goal: generate samples from a target distribution $\pi(\theta)$ (typically the posterior).
- MCMC constructs a **Markov chain** $\theta_0, \theta_1, \dots, \theta_n$ where the next state depends only on the current state: $\theta_{n+1} \sim P(\cdot | \theta_n)$.
- The transition kernel $P(\theta' | \theta)$ is designed so that $\pi(\theta)$ is the **stationary distribution** of the chain.
- A sufficient condition is **detailed balance**:

$$\pi(\theta)P(\theta' | \theta) = \pi(\theta')P(\theta | \theta')$$

- If the chain is irreducible and aperiodic then $\theta_n \xrightarrow{d} \pi(\theta)$ as $n \rightarrow \infty$.
- Common MCMC algorithms: Metropolis, Metropolis–Hastings, Gibbs sampling, Hamiltonian Monte Carlo.

Metropolis and Metropolis–Hastings

- We now construct a transition kernel that satisfies detailed balance with respect to $\pi(\theta)$.
- Given current state θ_t , generate a candidate using a proposal distribution $q(\theta'|\theta_t)$.
- Accept the proposed move with probability

$$\alpha(\theta_t, \theta') = \min \left(1, \frac{\pi(\theta')q(\theta_t|\theta')}{\pi(\theta_t)q(\theta'|\theta_t)} \right)$$

This defines the **Metropolis–Hastings algorithm**.

- The **Metropolis algorithm** is a special case where the proposal is symmetric, $q(\theta'|\theta) = q(\theta|\theta')$, giving acceptance probability
$$\alpha(\theta_t, \theta') = \min \left(1, \frac{\pi(\theta')}{\pi(\theta_t)} \right).$$
- This acceptance rule ensures detailed balance $\pi(\theta)P(\theta'|\theta) = \pi(\theta')P(\theta|\theta')$, so $\pi(\theta)$ is the stationary distribution.

Challenges in Standard MCMC That Motivate Adaptation

- **Difficult proposal tuning** Performance depends heavily on the choice of proposal distribution $q(y|x)$, but good tuning parameters are rarely known in advance.
- **Unknown covariance structure** Posterior parameters may be strongly correlated, yet this dependence structure is typically unknown before sampling begins.
- **Poor mixing from badly scaled proposals** Small proposal steps lead to slow exploration, while large steps lead to frequent rejection.
- **High-dimensional parameter spaces** As dimension increases, selecting appropriate proposal scales and covariance structures becomes increasingly difficult.

Example: Correlated Parameters

Suppose the posterior distribution

$$\pi(\theta_1, \theta_2)$$

has strong correlation between parameters.

The high-density region forms a long narrow ridge.

Random walk Metropolis often uses an isotropic proposal

$$Y = X + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I)$$

which moves equally in all directions.

Problem

- Proposals frequently move away from the ridge
- Many proposals are rejected
- The chain explores the posterior very slowly

Example: Proposal Scale

Random walk Metropolis proposes

$$Y = X + \epsilon, \quad \epsilon \sim N(0, \sigma^2)$$

where σ^2 controls the step size.

If σ^2 is too small

- Almost every proposal is accepted
- Moves are extremely small
- The chain explores the space very slowly

If σ^2 is too large

- Proposals frequently land in low-density regions
- Most proposals are rejected
- The chain remains stuck for long periods

Definition of Adaptive MCMC

- **Adaptive MCMC** refers to MCMC algorithms where the proposal distribution is updated during sampling using information from past samples.
- At iteration n , proposals are drawn from $q_n(y \mid x)$, where parameters of the proposal (such as scale or covariance) are allowed to change over time.
- A generic update is $Y_n \sim q_n(\cdot \mid X_{n-1})$, where the proposal q_n depends on the history $\{X_0, X_1, \dots, X_{n-1}\}$.
- **Key idea:** the algorithm learns better proposal parameters during sampling to improve efficiency while maintaining the correct stationary distribution.

Conditions for Adaptive MCMC Validity

Adaptive MCMC changes the transition kernel during sampling, so the chain is no longer a standard Markov chain.

Diminishing Adaptation

- The transition kernels P_{γ_n} change less over time.
- $\sup_{\theta} \|P_{\gamma_{n+1}}(\theta, \cdot) - P_{\gamma_n}(\theta, \cdot)\| \rightarrow 0$
- If adaptation diminishes, the chain eventually behaves like a Markov chain with an almost fixed kernel.
- This allows standard ergodic convergence arguments to apply asymptotically.

Containment Condition

- Prevents the algorithm from adapting toward proposal distributions that mix extremely slowly.
- Intuitively, the chain should not wander into regions where proposals become badly scaled and exploration becomes extremely slow.

Evaluating MCMC Performance

The goal of MCMC is to produce samples that explore the target distribution efficiently.

Effective Sample Size (ESS)

$$ESS = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}$$

- ρ_k is the autocorrelation at lag k which measures how similar the chain is to itself k iterations later; high autocorrelation indicates inefficient sampling.
- Measures how many **independent samples** the chain is equivalent to
- Larger ESS $\Rightarrow$ more efficient sampling

Autocorrelation

- Measures how strongly successive samples are related
- Slow decay $\Rightarrow$ chain moves slowly through the space
- Fast decay $\Rightarrow$ better exploration

Adaptive MCMC expectation

Adaptive algorithms $\rightarrow$ lower autocorrelation and larger ESS

Optimal Acceptance Rate

For random-walk Metropolis, theory suggests an efficient acceptance rate of

$$\alpha^* \approx 0.234$$

with proposal variance

$$\sigma^2 \approx \frac{2.38^2}{d}$$

where d is the dimension.

Interpretation

- Acceptance too high $\rightarrow$ steps are too small
- Acceptance too low $\rightarrow$ steps are too large
- Rates near 0.2–0.3 often indicate good proposal scaling

Adaptive MCMC methods adjust the proposal automatically to move toward this range.

Adaptive Metropolis Algorithm (Haario et al., 2001)

- The **Adaptive Metropolis (AM)** algorithm adapts the proposal covariance using the history of the chain.
- The proposal follows a random-walk form

$$\theta^{(n+1)} = \theta^{(n)} + \mathcal{N}(0, S_n)$$

where S_n is the proposal covariance matrix.

Algorithm

- 1 Initialize with covariance matrix S_0 .
- 2 Generate proposals using the Metropolis random walk proposal above.
- 3 Update the covariance using past samples:

$$S_n = \frac{1}{n} \sum_{i=1}^n (\theta^{(i)} - \bar{\theta})(\theta^{(i)} - \bar{\theta})^T + \epsilon I$$

where $\bar{\theta}$ is the empirical mean and ϵI ensures numerical stability.

Does Adaptive Metropolis Satisfy the Validity Conditions?

A common sufficient route to validity (ergodicity / LLN) for adaptive MCMC is: *Diminishing Adaptation* + *Containment* (a.k.a. *Bounded Convergence*).

Diminishing Adaptation: *Typically yes for AM.* If the proposal covariance is updated via empirical moments, then $\Sigma_{n+1} - \Sigma_n = O(1/n)$, so the Metropolis kernel changes less and less over time.

Containment / Bounded Convergence: *Not automatic.* It must be proved from stability assumptions, e.g.

- compact state space / bounded support; **or**
- sufficiently light target tails (plus regularity) *and* proposal safeguards (e.g. a fixed-mixture component, ϵI regularization, or truncation/projection).

Takeaway: AM is *designed* to make adaptation vanish, but ergodicity is *theorem-dependent*: it hinges on whether containment can be established for the target and the chosen stabilizers.

Checking Adaptive MCMC Assumptions in Practice

Diminishing Adaptation

- Examine **kernel change**, which measures how much the sampling rule changes between iterations.
- In adaptive Metropolis the kernel is determined by the proposal covariance, so kernel change reflects how much the proposal covariance changes from iteration n to $n + 1$.
- If kernel changes become much smaller later in the run than early in the run, the transition rule is stabilizing.

Containment

- Examine whether the proposal covariance remains well behaved.
- **Average marginal standard deviation** measures the typical proposal step size. If it stays bounded, the proposal scale is not exploding or collapsing.
- **Condition number** measures how stretched the covariance matrix is (ratio of largest to smallest eigenvalue).
- If it stabilizes at moderate values, the proposal geometry is not becoming extremely distorted.

Robust Adaptive Metropolis (RAM)

RAM adapts the proposal covariance to maintain a desired acceptance rate.

- ➊ **Proposal** Draw $U_n \sim q(\cdot)$ and propose $Y_n = X_{n-1} + S_{n-1}U_n$.
- ➋ **Accept/Reject** Compute $\alpha_n = \min\left(1, \frac{\pi(Y_n)}{\pi(X_{n-1})}\right)$. Set $X_n = Y_n$ if accepted, otherwise $X_n = X_{n-1}$.
- ➌ **Adapt Proposal** Update the proposal factor $S_n S_n^T = S_{n-1} \left(I + \eta_n (\alpha_n - \alpha^*) \frac{U_n U_n^T}{\|U_n\|^2} \right) S_{n-1}^T$.

Parameters

- $I : d \times d$ identity matrix
- η_n : learning rate
- α^* : target acceptance rate

Why RAM instead of Adaptive Metropolis?

- Directly targets an optimal acceptance rate
- More stable in high dimensions
- Avoids storing and recomputing full empirical covariance

Delayed Rejection Adaptive Metropolis (DRAM)

DRAM combines **Delayed Rejection** with **Adaptive Metropolis**.

- First proposal: $Y_1 \sim N(X_n, S_n)$.
- Accept with probability $\alpha_1 = \min(1, \pi(Y_1)/\pi(X_n))$.
- If rejected, attempt a **second proposal** $Y_2 \sim N(X_n, \gamma S_n)$ with $0 < \gamma < 1$.
- The second-stage acceptance probability corrects for the rejected first move:
$$\alpha_2 = \min\left(1, \frac{\pi(Y_2)q_1(Y_1|Y_2)(1-\alpha_1(Y_2, Y_1))}{\pi(X_n)q_1(Y_1|X_n)(1-\alpha_1(X_n, Y_1))}\right).$$
- The proposal covariance is then updated adaptively using past samples:
$$S_n \leftarrow \text{Cov}(X_1, \dots, X_n) + \epsilon I.$$

Why the second stage helps

- If a large move is rejected, a smaller “rescue” proposal is attempted.
- This reduces wasted iterations and improves exploration.

Examples of Adaptive MCMC Methods

Adaptive MCMC algorithms modify proposal parameters during sampling to improve efficiency while preserving convergence to the target distribution.

- **Adaptive Metropolis (AM)** Adapts the proposal covariance using the empirical covariance of past samples.
- **Robust Adaptive Metropolis (RAM)** Updates the proposal covariance to target a desired acceptance rate.
- **Delayed Rejection Adaptive Metropolis (DRAM)** Combines adaptive covariance learning with secondary proposals after rejection.
- **Adaptive Metropolis-within-Gibbs** Adapts proposal scales within Gibbs sampling updates.
- **Adaptive Gibbs Sampling** Adapts blocking structure or update probabilities.
- **Adaptive Independent Metropolis** Adapts a global proposal distribution.
- **Adaptive Hamiltonian Monte Carlo / NUTS** Tunes step size and mass matrix during warmup.

Thank You